

is a dynamic game where the **probability** , and the game transitions through different "states" over time

To connect it to your previous question: the Minimax tree you shared is

. If you choose to move from node A to node B, you end up at B with 100% certainty. In a stochastic game, choosing an action might give you a 70% chance of moving to state B, and a 30% chance of moving to state C. (like rolling a die or drawing a card)

Here is a breakdown of how they work and why they matter in AI:

Key Components of a Stochastic Game

Formally introduced by Lloyd Shapley in 1953, a stochastic game consists of:

1. **State**: The current situation or environment of the game (like a specific board layout)
2. **Players**: Two or more agents making decisions
3. **Actions**: The moves available to each player in a given state
4. **Transitions**: The mathematical chance of moving to a new state based on the current state and the actions chosen by all players
5. **Payoffs**: The points or utility a player receives at each stage

How it Differs from Standard Games (like Chess)

- **Deterministic**: Perfect information, no randomness. Minimax works perfectly here
- **Stochastic**: Involve chance (dice, shuffled cards) or hidden information. The environment itself can also change randomly (e.g., a robot slipping on a wet floor).

Why are they important in AI?

Standard Minimax fails in stochastic games because you can't just calculate a single guaranteed path to victory. Instead, AI uses stochastic games to model real-world uncertainty. They are the foundation for:

1. **Multi-Agent Systems**: When multiple AI agents learn to cooperate or compete in an unpredictable environment (like self-driving cars navigating traffic, or AI playing real-time strategy games)
2. **Expectimax**: This is a variation of Minimax used for stochastic games. It adds "Chance Nodes" (usually drawn as circles in a game tree) where the algorithm calculates the expected value (average outcome) of a random event rather than just picking the max or min.

A Simple Example

Imagine a game where you and an opponent are trying to reach a finish line.

- You roll a die to move forward (the **stochastic/random** part)
- However, there are also "trap" squares. If you land on one, the opponent gets to choose whether to send you back 2 spaces or 3

Because the dice roll is random, but the opponent's choice is strategic, this is a stochastic game. The AI has to calculate the best strategy that maximizes its average reward over many turns, rather than guaranteeing a win on a single turn.

Pseudo Code:

Initialize current_state (e.g., Start)

Set game_over = False

While game_over is False:

1. Observe current_state

2. Player 1 chooses Action_A

3. Player 2 chooses Action_B

4. Look up Transition_Probabilities for (current_state, Action_A, Action_B)

// Yeh step game ko "stochastic" banata hai.

// Example: 70% chance State 2 ka, 30% chance State 1 ka.

5. Roll a random dice/number to pick the actual next_state based on those probabilities

6. Calculate Reward_1 and Reward_2 based on the transition

7. Update current_state = next_state

8. If current_state is a Terminal State (End):

game_over = True

Print Final Rewards